

OpenVAT — Runtime Animation API

Developer Documentation

1. Overview

This document covers all runtime API additions made to the OpenVAT system (original author: Luke Stilson). The additions focus on three areas:

- Pause / Resume / Stop controls on VATController and VATAnimStateMachine
- Real-time playback speed control
- Multi-object sequencing via VATSequencer

All time control is achieved by sliding the shader time offsets (`_TimeOffsetA`, `_TimeOffsetB`, `_BlendStartTime`) on the CPU. The shader itself is not modified.

2. Architecture

The system is composed of four classes:

Class	Role
VATController	MonoBehaviour facade. Public API entry point. Attach to each VAT GameObject.
VATAnimStateMachine	Pure C# state machine. Manages clip indices, blend timers, and time offsets.
VATMaterialBinder	Pushes VATMaterialState to the Renderer material (unchanged).
VATSequencer	MonoBehaviour coordinator. Manages ordered activation of multiple VAT GameObjects.

3. VATController

Attach one VATController per VAT GameObject. It owns a VATAnimStateMachine and a VATMaterialBinder, and exposes the full playback API.

3.1 Playback

Method / Property	Returns	Description
<code>PlayIndex(int index, float transition = 0.25f)</code>	void	Play clip at index with optional crossfade duration (seconds).

<code>Play(string name, float transition = 0.25f)</code>	void	Play clip by name with optional crossfade.
<code>PlayInstant(int index)</code>	void	Play clip at index with no transition (cut).
<code>PlaySequence(int min, int max, float transition = 0.25f)</code>	void	Loop clips between min and max indices using the GPU sequence path.

3.2 Pause / Resume / Stop

Method / Property	Returns	Description
<code>Pause()</code>	void	Freeze the animation on its current frame. Safe to call multiple times.
<code>Resume()</code>	void	Resume from the exact frame it was paused on.
<code>TogglePause()</code>	void	Toggle between Pause and Resume.
<code>Stop(int index = -1)</code>	void	Stop and rewind to the start of the given clip index. Defaults to singleAnimIndex if -1.
<code>IsPaused</code>	bool (get)	True while the animation is paused.

⚠ Calling any Play() method while paused automatically calls Resume() first so time offsets are consistent.*

3.3 Speed

Method / Property	Returns	Description
<code>Speed</code>	float (get/set)	Playback speed multiplier. 1.0 = normal, 0.5 = half speed, 2.0 = double. Clamped to >= 0. Setting to 0 freezes visually but is not equivalent to Pause().

⚠ Speed = 0 freezes the visual frame but does not preserve the resume position. Use Pause() / Resume() for a freeze that resumes at the correct frame.

4. VATAnimStateMachine

Instantiated internally by VATController. You do not need to use this class directly unless you are building a custom controller. All public members below are accessible through VATController.Speed and VATController.IsPaused.

Key fields exposed for advanced use:

Field	Type	Description
-------	------	-------------

speedMultiplier	float	Scales fpsA and fpsB before they are sent to the shader. Prefer VATController.Speed.
IsPaused	bool	Read-only. True while paused.

4.1 How pause works

The shader computes the current frame as:

```
frame_time = _Time.y - _TimeOffsetA
```

`_Time.y` is Unity's built-in GPU clock and cannot be stopped. While paused, `VATAnimStateMachine` slides `_TimeOffsetA` (and `_TimeOffsetB`, `_BlendStartTime`) forward at the same rate as `Time.time` every frame, so that the difference stays constant and the shader sees a frozen frame. On `Resume()`, the offsets are already correct and sliding stops.

5. VATSequencer

Manages an ordered list of VAT GameObjects representing discrete states (e.g. flood simulation steps). Each step activates one GameObject and deactivates the previous one. Progression is entirely manual — driven by your external game script.

5.1 VATStep struct

Field	Type	Description
vat	VATController	The VATController on the target GameObject.
animIndex	int	Clip index to play when this step activates.
transition	float	Crossfade duration in seconds for <code>PlayIndex()</code> .

5.2 Public API

Method / Property	Returns	Description
<code>StartSequence()</code>	void	Deactivates all step GameObjects, resets index to -1, fires <code>OnSequenceStart</code> , then activates step 0.
<code>NextStep()</code>	void	Deactivates the current step, activates the next one. Fires <code>OnSequenceEnd</code> if already on the last step.
<code>GoToStep(int index)</code>	void	Jump directly to any step. Deactivates the current step, activates the target. Use for non-linear navigation.
<code>CurrentStepIndex</code>	int (get)	Index of the currently active step. -1 if not

		started.
IsStarted	bool (get)	True if StartSequence() has been called and at least one step is active.
IsFinished	bool (get)	True if NextStep() was called past the last step.
CurrentStep	VATStep? (get)	The currently active VATStep, or null if not started.

5.3 Events

VATSequencer exposes both UnityEvent (Inspector-bindable) and C# Action delegates for each event:

UnityEvent (Inspector)	C# Event	Fired when...
onSequenceStart	OnSequenceStart	StartSequence() is called.
onSequenceEnd	OnSequenceEnd	NextStep() is called past the last step.
onStepChanged(int, VATStep)	OnStepChanged(int, VATStep)	Any step becomes active (index + step passed as arguments).

6. Integration with an External Script

The typical integration pattern from your game manager script:

6.1 Minimal setup

```
// In your game manager - Awake or Start
[SerializeField] private VATSequencer _flooding;

void Start() {
    _flooding.OnSequenceEnd += HandleFloodComplete;
    _flooding.OnStepChanged += HandleStepChanged;
}
```

6.2 Triggering steps from dialogue / events

```
// First call - initialises and shows step 0
_flooding.StartSequence();

// Called each time the narrative progresses
_flooding.NextStep();
```

6.3 Non-linear navigation (future)

```
// Jump to a specific flood level directly
_flooding.GoToStep(3);
```

6.4 Reacting to step changes

```

void HandleStepChanged(int index, VATSequencer.VATStep step) {
    Debug.Log($"Flood level {index} active: {step.vat.name}");
    // e.g. trigger dialogue, UI update, audio...
}

```

6.5 Pausing / resuming during a cutscene

```

// Pause the active VAT during a dialogue
_flooding.CurrentStep?.vat.Pause();

// Resume after dialogue
_flooding.CurrentStep?.vat.Resume();

```

7. Editor Debuggers

Two Editor-only debugger components are provided. Both require Play Mode.

Component	Script file	Target
VATDebugger	VATDebuggerEditor.cs (in Editor/)	Single VATController — pause, resume, stop, speed slider.
VATSequencerDebugger	VATSequencerDebuggerEditor.cs (in Editor/)	VATSequencer — start, next step, jump to any step by name.

*⚠ Both Editor scripts must be placed inside a folder named **Editor/** to prevent inclusion in builds.*

8. Known Limitations

- **Speed vs Pause** — Speed = 0 is not equivalent to Pause().
- The GPU sequence path (PlaySequence) is driven entirely by the shader. Pause/Resume work correctly for it, but Speed only affects the framerate of the last clip in the sequence, not the inter-clip timing.
- **Deactivation** — VATSequencer.NextStep() does not auto-stop the VAT animation on the deactivated GameObject. The GameObject is simply disabled, which stops rendering. If you need the animation timer to reset, call Stop() on the previous step's VATController before calling NextStep().

9. Modified Files

File	Changes
VATController.cs	Added Update(), Pause(), Resume(), Stop(), TogglePause(), IsPaused, Speed property. Fixed PlayInstant() transition bug (0.25f → 0f).
VATAnimStateMachine.cs	Added Pause(), Resume(), Stop(), IsPaused, speedMultiplier.

	UpdateAndGetState() now slides offsets during pause. All Play*() methods call Resume() automatically.
VATSequencer.cs	New file. Replaces coroutine-based version with manual step control.
VATSequencerDebugger.cs	New file. Runtime Inspector for VATSequencer.
VATDebuggerEditor.cs	Added Speed slider, shortcut buttons, and Play Mode guard.